

Missouri University of Science and Technology

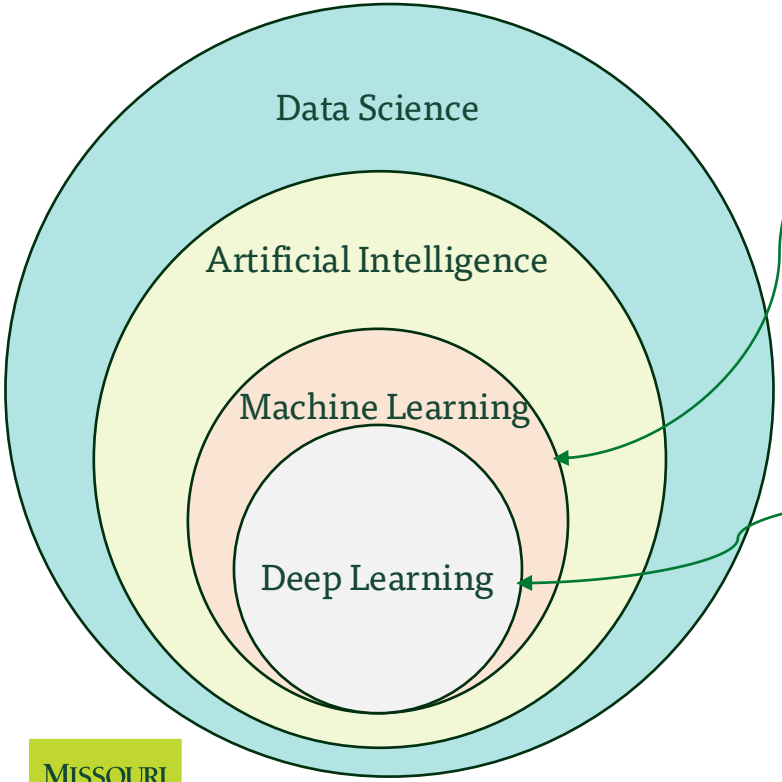
Intro to Data Science
CS 3402 (Spring 2026)

Lecture 7 Supervised Learning

Xiaowei Yu, Ph.D.

Department of Computer Science

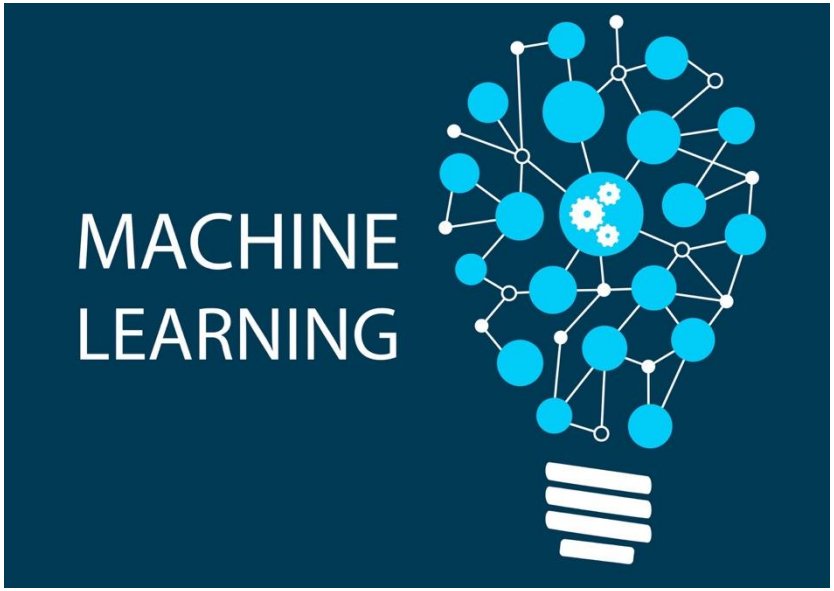
DS/AI/ML/DL

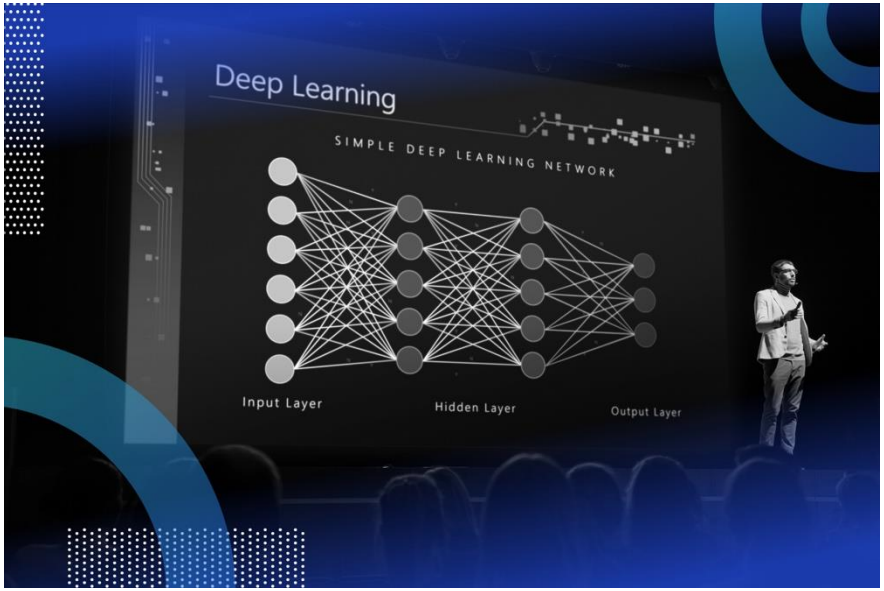


Machine Learning
- Supervised Learning (Labeled historical data)
- Unsupervised Learning (Unlabeled data)
- Reinforced Learning

Deep Learning
- Multilayer Perceptron Neural Network (MLP NN)
- Convolutional NN
- Recursive NN
- Recurrent NN
- Sequence to Sequence NN
- Transformer NN
- …

What is Learning?





Toy Example of Gradient Descent in ML Models

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

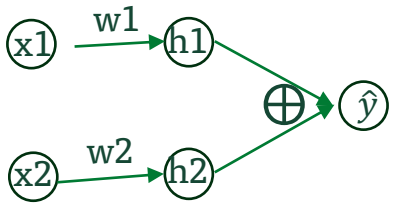
Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

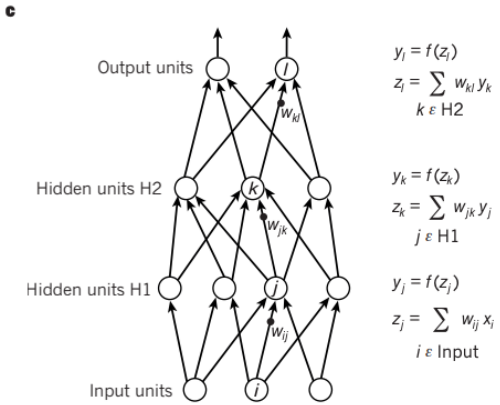
Learning rate: 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction:

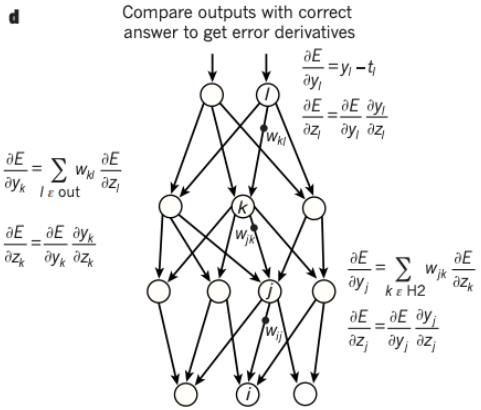
$\hat{y} = w_1 m + w_2 n$





Forward pass

Using existing weights for prediction



Back propagation

Updating weights via loss function

Toy Example of Gradient Descent in ML Models

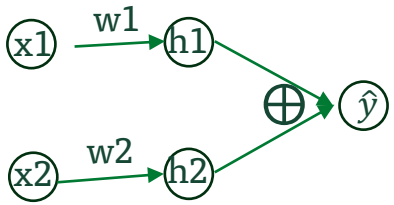
Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

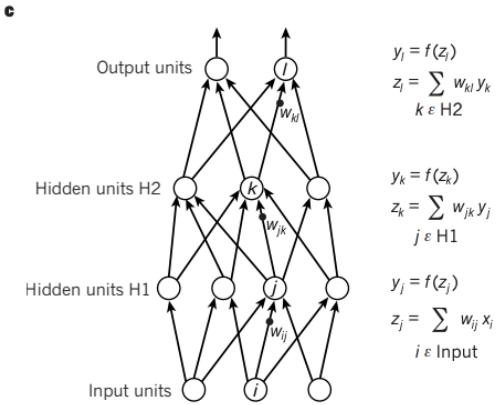
Learning rate: 0.1

Initial weights: $w1 = 1, w2 = 1$

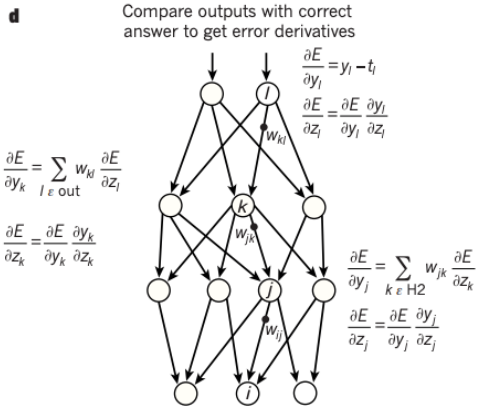


Step 2: Compute the Loss

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$$



Forward pass



Back propagation

Gradient Chain Rule

Gradient Chain Rule

If

$$L = L(\hat{y}), \quad \hat{y} = f(\mathbf{w}, \mathbf{x})$$

Then for each weight w_j:

$$\boxed{\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_j}}$$

This is the **gradient chain rule**.

Model

$$\hat{y} = w_1 x_1 + w_2 x_2$$

Loss

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Step-by-step chain rule

Step 1: Loss → prediction

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

Step 2: Prediction → weights

$$\frac{\partial \hat{y}}{\partial w_1} = x_1, \quad \frac{\partial \hat{y}}{\partial w_2} = x_2$$

Step 3: Chain them

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2$$

That's exactly what we used in gradient descent.

Gradient Chain Rule

Model

$$\hat{y} = w_1 x_1 + w_2 x_2$$

Loss

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Step-by-step chain rule

Step 1: Loss → prediction

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

Step 2: Prediction → weights

$$\frac{\partial \hat{y}}{\partial w_1} = x_1, \quad \frac{\partial \hat{y}}{\partial w_2} = x_2$$

Step 3: Chain them

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2$$

That's exactly what we used in gradient descent.

Step 4: Substitute related terms into results.

$$\frac{\partial L}{\partial w_1} = (w_1 x_1 + w_2 x_2 - y)x_1$$

$$\frac{\partial L}{\partial w_2} = (w_1 x_1 + w_2 x_2 - y)x_2$$

Toy Example of Gradient Descent in ML Models

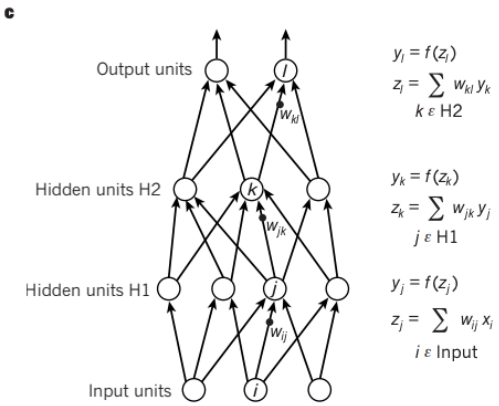
Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

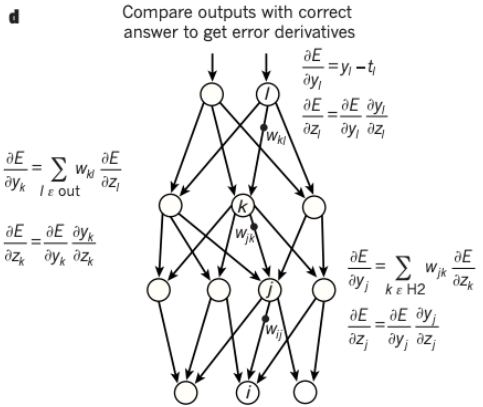
One Training sample: (m, n, y1)

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: $w1 = 1, w2 = 1$





Step 3: Compute the gradients

Forward pass

Back propagation

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \frac{1}{2}(\hat{y} - y1)^2}{\partial w_1} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_1} = (w_1 m + w_2 n - y1)\, m$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \frac{1}{2}(\hat{y} - y1)^2}{\partial w_2} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_2} = (w_1 m + w_2 n - y1)\, n$$

Toy Example of Gradient Descent in ML Models

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (m, n, y1)

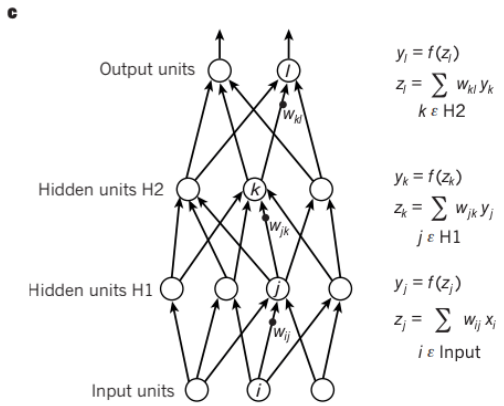
Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

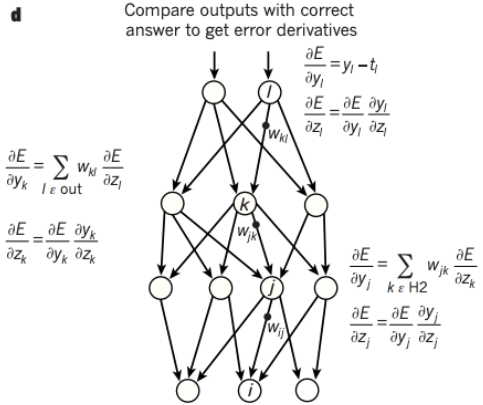
Initial weights: $w1 = 1, w2 = 1$

Step 4: Gradient descent update rule

$$w_i^{new} \leftarrow w_i^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$



Forward pass



Back propagation

$$w_1^{new} = w_1^{old} - 0.1 * (w_1 m + w_2 n - y1)\, m$$

$$w_2^{new} = w_2^{old} - 0.1 * (w_1 m + w_2 n - y1)\, n$$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: $(2, 1, 4)$

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: $(2, 1, 4)$

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: w1 = 1, w2 = 1

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^{\wedge}2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial (\hat{y} - y1)^2}{\partial w_1} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_1} = (w_1 m + w_2 n - y1)\, m$$

$$= (1*2 + 1*1 - 4) * 2$$

$$= -2$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial (\hat{y} - y1)^2}{\partial w_2} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_2} = (w_1 m + w_2 n - y1) n$$

$$= (1 * 2 + 1*1 - 4) * 1$$

$$= -1$$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: $(2, 1, 4)$

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = -2 \qquad \frac{\partial \mathcal{L}}{\partial w_2} = -1$$

Step 4: Gradient descent update rule

$$w_i^{new} \leftarrow w_i^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$

$$w_1^{new} = w_1^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_1} = 1 - 0.1 * (-2) = 1.2 \qquad w_2^{new} = w_2^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_2} = 1 - 0.1 * (-1) = 1.1$$

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = -2 \qquad \frac{\partial \mathcal{L}}{\partial w_2} = -1$$

Step 4: Gradient descent update rule

$w_1^{new} = 1.2$ $w_2^{new} = 1.1$

Does the updated weights help the model better fit the data?

Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

Model: $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

One Training sample: (2, 1, 4)

Loss function (squared error): $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Learning rate η *:* 0.1

Initial weights: $w1 = 1, w2 = 1$

Previous Model: $\hat{y} = 1.0 * x_1 + 1.0\ x_2$

Previous Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

Current Model: $\hat{y} = 1.2 * x_1 + 1.1\ x_2$

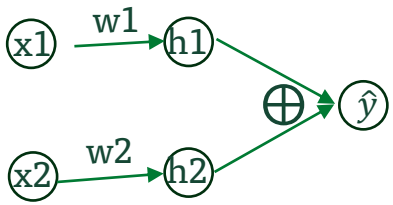
Current Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 1/2 (1.2 * 2 + 1.1 * 1 -4)^2
= **0.125**

Does the updated weights help the model better fit the data?

Toy Example of Gradient Descent in ML Models

Gradient descent helps the neural networks fit the data better by minimizing the loss function.

2 parameters



- More parameters?
- More iterations?
- Increasing/decreasing learning rate?
- Different loss functions?
- ….

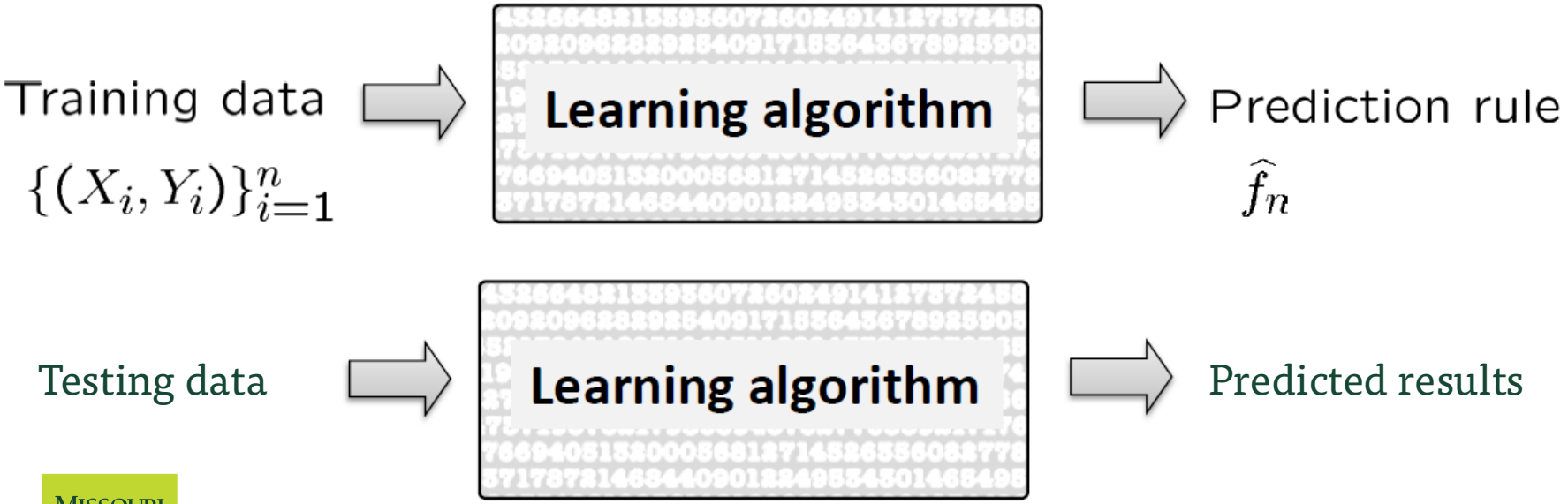
Parameters of LLMs

Organization	Model	Parameters	Notes
OpenAI	GPT-2 (small)	124M	Fully disclosed
	GPT-2 (medium)	355M	
	GPT-2 (large)	774M	
	GPT-2 (XL)	1.5B	
	GPT-3	175B	Official
	GPT-3.5	~175B	Same scale as GPT-3
	GPT-4	Undisclosed	Estimates range 1T+ (MoE suspected)
	GPT-4o	Undisclosed	No official numbers
Anthropic	Claude 1	Undisclosed	Larger than GPT-3 class
	Claude 2	Undisclosed	Likely hundreds of billions
	Claude 3 (Opus)	Undisclosed	Very large, MoE-style
Google / DeepMind	PaLM	540B	Official
	PaLM 2	Undisclosed	Smaller + more efficient
	Gemini 1.0 Ultra	Undisclosed	Competes with GPT-4
	Gemini 1.5 Pro	Undisclosed	Very long context
Meta (Facebook)	LLaMA	7B	Official
	LLaMA	13B	
	LLaMA	33B	
	LLaMA	65B	
Mistral AI	Mistral 7B	7B	Dense
	Mixtral 8×7B	~47B	MoE (13B active per token)
	Mixtral 8×22B	~176B	MoE
xAI	Grok-1	~314B	Open weights
AI21 Labs	Jurassic-1 Jumbo	178B	GPT-3 class
Cohere	Command	~52B	Estimated
Tencent	Hunyuan	100B+	Chinese LLM
Alibaba	Qwen	7B–72B	Multiple sizes

Need massive parallel computing hardware, such as GPUs, TPUs, to train LLMs.

Supervised Learning

Definition: Algorithms which learn from a training set of labeled examples (exemplars) to generalize to the set of all possible inputs.



Supervised Learning Tasks

Visual Object Categorization



We are given categories for these images:

From ETH database of object categories, [Leibe & Schiele 2003]



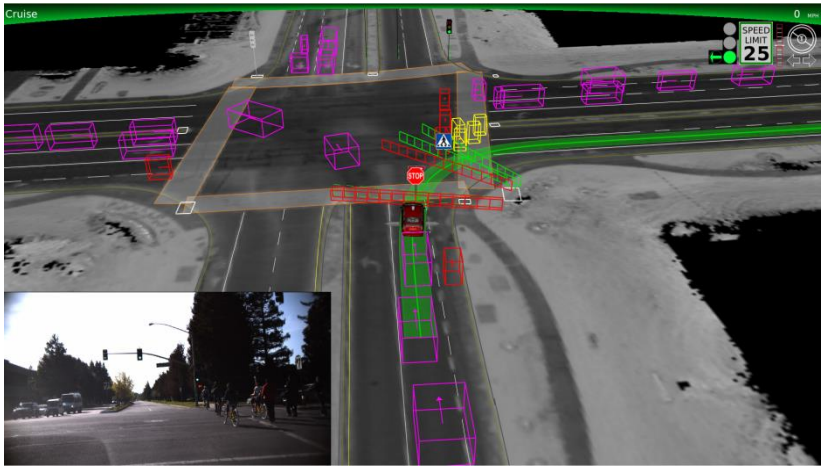
What are these?

A classification problem: predict category y based on image x.

Supervised Learning Tasks

Self-driving





Supervised Learning Tasks

Text Classification



Company home page

vs

Personal home page

vs

University home page

vs

...

How to Solve Tasks from DS/AI perspective?

1. Data Collection & Preprocessing (Preparation, Check, Statistics, Visualization)

2. Model/Algorithm Choice (Neural Networks, SVM, KNN, Logistic Regression, etc.)

3. Training (Loss function design, hyperparameters optimization, etc.)

4. Performance Monitoring/Evaluation (Performance Metrics, Comparison with baselines)

Supervised Learning Example Problem



Wine Quality

Donated on 10/6/2009

Two datasets are included, related to red and white vinho verde wine samples, from the north of Portugal. The goal is to model wine quality based on physicochemical tests (see [Cortez et al., 2009], http://www3.dsi.uminho.pt/pcortez/wine/).

Dataset Characteristics	Subject Area	Associated Tasks
Multivariate	Business	Classification, Regression
Feature Type	**# Instances**	**# Features**
Real	4898	11

Dataset Information

Additional Information

The two datasets are related to red and white variants of the Portuguese "Vinho Verde" wine. For more details, consult: http://www.vinhoverde.pt/en/ or the reference [Cortez et al., 2009]. Due to privacy and logistic issues, only physicochemical (inputs) and sensory (the output) variables are available (e.g. there is no data about grape types, wine brand, wine selling ...

SHOW MORE

Has Missing Values?

No

Dataset site: https://archive.ics.uci.edu/dataset/186/wine+quality

SL Example Problem – Wine Quality Classification

winequality-red

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
7.4	0.7	0	1.9	0.076	11	34	0.9978	3.51	0.56	9.4	5
7.8	0.88	0	2.6	0.098	25	67	0.9968	3.2	0.68	9.8	5
7.8	0.76	0.04	2.3	0.092	15	54	0.997	3.26	0.65	9.8	5
11.2	0.28	0.56	1.9	0.075	17	60	0.998	3.16	0.58	9.8	6
7.4	0.7	0	1.9	0.076	11	34	0.9978	3.51	0.56	9.4	5
7.4	0.66	0	1.8	0.075	13	40	0.9978	3.51	0.56	9.4	5
7.9	0.6	0.06	1.6	0.069	15	59	0.9964	3.3	0.46	9.4	5
7.3	0.65	0	1.2	0.065	15	21	0.9946	3.39	0.47	10	7
7.8	0.58	0.02	2	0.073	9	18	0.9968	3.36	0.57	9.5	7
7.5	0.5	0.36	6.1	0.071	17	102	0.9978	3.35	0.8	10.5	5
6.7	0.58	0.08	1.8	0.097	15	65	0.9959	3.28	0.54	9.2	5
7.5	0.5	0.36	6.1	0.071	17	102	0.9978	3.35	0.8	10.5	5
5.6	0.615	0	1.6	0.089	16	59	0.9943	3.58	0.52	9.9	5
7.8	0.61	0.29	1.6	0.114	9	29	0.9974	3.26	1.56	9.1	5
8.9	0.62	0.18	3.8	0.176	52	145	0.9986	3.16	0.88	9.2	5

Csv files

SL Example Problem – Wine Quality Classification

Read data files

```
import pandas as pd

df = pd.read_csv("winequality-red.csv", sep=";")
print(df.head())
```

```
print(df.shape)      # (1599, 12)
print(df.columns)
print(df.info())
```

winequality-red.csv and winequality-white.csv

SL Example Problem – Wine Quality Classification

Check, Statistics Metrics, Visualization, Preprocessing

Input data should be converted into approximate format for model training.

```
df["label"] = (df["quality"] >= 6).astype(int)

X = df.drop(columns=["quality", "label"])
y = df["label"]
```

SL Example Problem – Wine Quality Classification

```
df["label"] = (df["quality"] >= 6).astype(int)

X = df.drop(columns=["quality", "label"])
y = df["label"]
```

The size of the dataset is expected to be

[n_samples, d_features]:

- n_samples: The number of samples (n)
- n_features: The number of features (d) or distinct traits that can be used to describe each item in a quantitative manner

$$\text{feature matrix}: \quad \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ x_{31} & x_{32} & \cdots & x_{3d} \\ \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

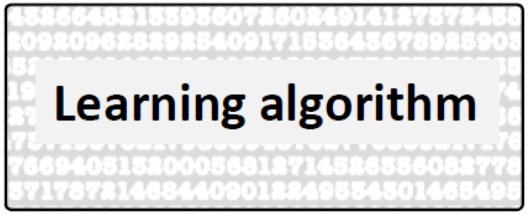
$$\text{label vector}: \quad \mathbf{y}^T = [y_1, y_2, y_3, \cdots y_n]$$

!Textbook can have errors

SL Example Problem – Wine Quality Classification

Training data

$\{(X_i, Y_i)\}_{i=1}^{n}$



Prediction rule

$\widehat{f}_n$

Regression Model? Neural Networks? .....

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

SL Example Problem – Wine Quality Classification

Performance Monitoring/Evaluation

$$\text{Accuracy} = \frac{Correct\ Prediction}{Total\ number\ in\ test\ set}$$

What else? (Leave this as your homework)

Syllabus needs to be updated

We observe Spring break from 03/22/2026 – 03/30/2026.

SPRING SEMESTER 2026

Open Registration Ends	January 19, Monday
Martin Luther King, Jr. Recognition Holiday	January 19, Monday
Spring semester opens 8:00 a.m.	January 20, Tuesday
Classwork begins 8:00 a.m.	January 20, Tuesday
Spring Recess begins 8:00 a.m.	March 12, Thursday
Mid-Semester	March 14, Saturday
Spring Recess ends 8:00 a.m.	March 16, Monday
Spring Break begins 8:00 a.m.	March 22, Sunday
Spring Break ends 8:00 a.m.	March 30, Monday
Last Class Day	May 8, Friday

The corresponding content will be moved forward.

Group Project Topic 1

Question: Are general-purpose pretrained models better than task-specific models for medical datasets?

Intro: In recent years, large **pretrained foundation models**, such as LLMs, VLMs, have transformed machine learning by demonstrating strong generalization across a wide range of tasks, often outperforming models trained from scratch even when labeled data are limited. This success has motivated growing interest in applying general-purpose pretrained models to **medical and healthcare data**, where annotation is expensive and datasets are often small.

Group Project Topic 1

At the same time, many medical machine learning applications have traditionally relied on **task-specific model designs** that incorporate domain knowledge, such as structured features, biologically motivated representations, or specialized network architectures. These models embed strong **inductive biases** tailored to particular diseases, modalities, or data structures, which can be especially valuable in noisy and heterogeneous clinical datasets.

Whether pretrained foundation models consistently outperform task-specific models in medical applications remains an open and actively debated question. Medical datasets differ substantially from natural language or image benchmarks: they are typically smaller, highly structured, multimodal, and closely tied to underlying biological mechanisms. As a result, it is unclear when the benefits of large-scale pretraining outweigh the advantages of carefully designed, domain-specific modeling choices.

Group Project Topic 1

In this project, you will explore this question by **comparing task-specific designed models with pretrained or general-purpose foundation models across multiple medical datasets spanning different diseases and data modalities**. You will select medical datasets (**at least two**) and evaluate models tailored explicitly to the dataset alongside pretrained models adapted through transfer learning. The goal is to systematically assess their **predictive performance, robustness, and interpretability**, and to understand under what conditions domain-specific inductive bias may still outperform scale and pretraining.

Through these comparisons, you will gain hands-on experience with modern medical machine learning pipelines and develop critical insight into the tradeoffs between specialization and generalization in real-world healthcare applications.

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

1. Data Collection and Preprocessing

You should select **two or more medical datasets**, potentially spanning different diseases or data modalities (e.g., medical images, clinical tabular data, signals, or text). Public sources such as **Kaggle**, **UCI Machine Learning Repository**, or other reputable open medical datasets may be used.

Dataset description: Disease; Data modality (image, tabular, time series, text, etc.); Source of the dataset
Data inspection and quality check: Identify missing values, incomplete records, or corrupted samples; Detect potential outliers or abnormal samples
Data preprocessing: Handle missing data (e.g., removal, imputation, masking); Normalize or standardize features if necessary; Resize, augment, or transform data for model compatibility (especially for images)
Dataset statistics: Total number of samples; Number of classes (if classification); Class distribution
Data splitting: Clearly describe how the data are split into training, validation, and test sets

All preprocessing choices should be **clearly documented and justified**.

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

2. Model Selection and Experimental Design

You must compare **pretrained foundation models** with **task-specific designed models** on the same dataset(s).

(a) Large Language Models (LLMs)

For datasets involving text/structured data that can be converted into text form:
Examples: **ChatGPT, Claude, Gemini**, or similar models
Learning settings:
Zero-shot learning: no task-specific examples provided
Few-shot learning: a small number of labeled examples provided as prompts
Clearly describe:
Prompt design
Number of examples used
Evaluation method

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

2. Model Selection and Experimental Design

You must compare **pretrained foundation models** with **task-specific designed models** on the same dataset(s).

(b) Pretrained Vision Models (if applicable)
For image-based medical datasets:
Examples: **ResNet**, **Vision Transformer (ViT)**, or other pretrained vision models
Learning settings:

- **Zero-shot or frozen-feature evaluation**
- **Fine-tuning**

Report:

- Training hyperparameters
- Performance Metrics

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

2. Model Selection and Experimental Design

You must compare **pretrained foundation models** with **task-specific designed models** on the same dataset(s).

(c) Task-Specific Designed Models

You should design and train **task-specific models from scratch**, tailored to the chosen dataset.
Possible model choices include (but are not limited to):
MLP for tabular or low-dimensional data
CNN-based architectures for image or signal data
Transformer-based architectures trained from scratch
Other reasonable architectures motivated by the dataset structure
You should:
Explain the motivation behind the model design
Train models using only the task-specific data
Apply appropriate loss function and validation strategies

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

3. Evaluation and Analysis

You should evaluate models using appropriate performance metrics, such as:
Accuracy, precision, recall, F1-score
ROC-AUC or PR-AUC (if applicable)
Other task-relevant metrics
Beyond raw performance, students are encouraged to analyze:
Robustness (e.g., sensitivity to hyperparameters, data size or noise)
Interpretability (feature importance, attention maps, or qualitative analysis)

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

4. Documentation, Reporting, and Presentation

4.1 Project Report

You must submit a written report that includes:

- Introduction and motivation
- Dataset description and preprocessing
- Model descriptions and experimental setup
- Results and comparisons
- Discussion and key findings
- Limitations and future directions
- All results should be clearly organized using tables and figures.

Topic 1- Task-Specific Models vs. Pretrained Foundation Models for Medical Data Analysis

4. Documentation, Reporting, and Presentation

4.2 Presentation Slides

You should prepare a slide presentation covering:

- Problem definition and dataset
- Model comparison strategy
- Key experimental results
- Main insights and conclusions
- Presentations should emphasize **what was learned**, not just which model performed best.

Peer Evaluation of Presentations

- In addition to instructor evaluation, each team's presentation will be **rated by other teams** based on its **overall quality, clarity, and organization**.
- Please ensure your slides are well-structured and easy to follow, Clear visuals and a logical flow are strongly encouraged.